

# Agentic AI System in Confidential Computing (Agent-CC)

- Revision: 0.8
- Song, Edmund (edmund.song@intel.com)

| Revision | Date       | Summary                          |
|----------|------------|----------------------------------|
| 0.6      | 2026-05-19 | Initial working draft.           |
| 0.7      | 2026-06-10 | Update for Agent-CC 1.0 release. |
| 0.8      | 2026-06-28 | Trust Guard update.              |

This document examines how agentic AI systems can be deployed using confidential computing with Intel TDX as the reference. Its central point is not simply where agents run, but how this model can provide stronger isolation, measurable execution, and verifiable trust for agent workloads. In this context, Agent-CC is defined as an Intel TDX-based deployment architecture and reference implementation for confidential AI agent workloads: it is designed to be agnostic for agent-framework and services, and combines runtime isolation, trusted execution controls, and service-to-service trust verification as one coherent system. Within this document, we focus on how these architecture principles are applied to agent workload deployment, with the detailed mechanisms and design trade-offs developed in the later sections.

Confidential computing alone is not assumed to solve all AI security concerns. Instead, confidential-computing deployment is treated as an enabling layer that, when combined with policy, isolation, verification, and secure service interaction, can help mitigate major risks in AI agent deployment.

This article is organized in two parts. Chapters 1 and 2 characterize agent deployment architectures and patterns, outline their security challenges, and explain the additional security value provided by confidential-computing deployment. Chapters 3 to 5 then present how Agent-CC with Intel TDX mitigates those challenges through three dimensions: lifecycle data protection, build-to-runtime integrity, and trusted service composition.

In confidential computing, a Trusted Execution Environment (TEE) is the core hardware-protected runtime mechanism that isolates code and data in use from the rest of the system, including higher-privilege software layers. In this article, TEE is used as the general concept, while TD (Trusted Domain) is used for the Intel TDX-specific realization of that concept, typically as a TDX-backed virtual machine (TDVM).

## 1. Agent Architecture

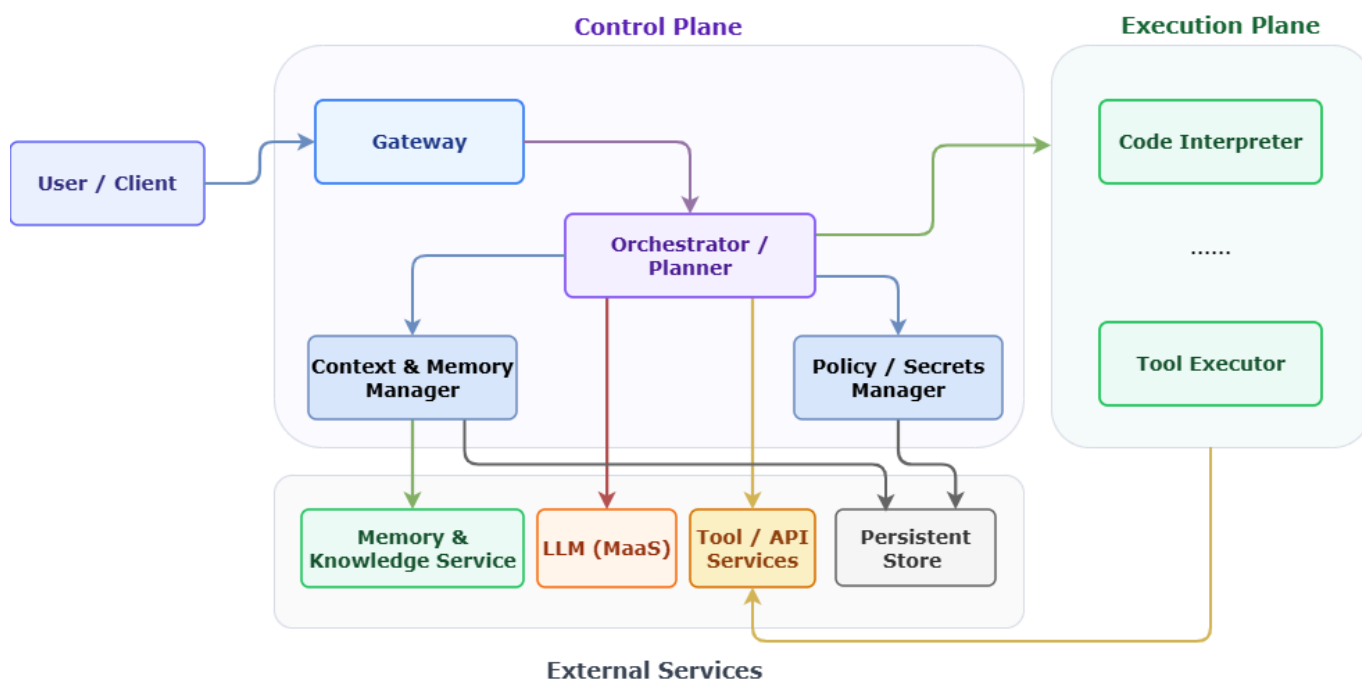
### 1.1 Deployment Architecture

To better analyze the security issues that arise during agent deployment, this document models its deployment architecture into three domains: orchestration (or Control Plane), Execution Plane, and Service Dependencies. This separation is useful in TEE-based systems because the placement of each function determines the isolation boundary, the attestation scope, and the operational model.

- **Control Plane:** governance and orchestration components that control admission, planning, policy, and secrets, including Gateway, Orchestrator / Planner, Context & Memory Manager, and Policy & Secrets Manager.
- **Execution Plane:** workload-execution components that run task logic and tool calls, typically under stronger runtime isolation, including Code Interpreter and Tool Executors.
- **External Services:** remote capability providers outside the local trusted boundary, including LLM / MaaS, Memory Backends & Knowledge Retrieval Service, and Persistent Store.

In practical terms, the Control Plane is usually the narrower and more trusted side. The Execution Plane is the danger zone, because it is the natural place for untrusted code, third-party tools, and dynamic workloads, and therefore usually requires an additional sandbox or stronger isolation boundary. External Services remain outside the local trusted boundary and are reached through controlled interfaces.

Given agent systems are open and evolving quickly, both the execution environment and the connected external services may change dynamically over time. This creates the same trust problem: new code, tools, policies, or service connections should not enter the running system without admission, measurement, and verification.



**Figure 1: Agent Deployment Architecture**

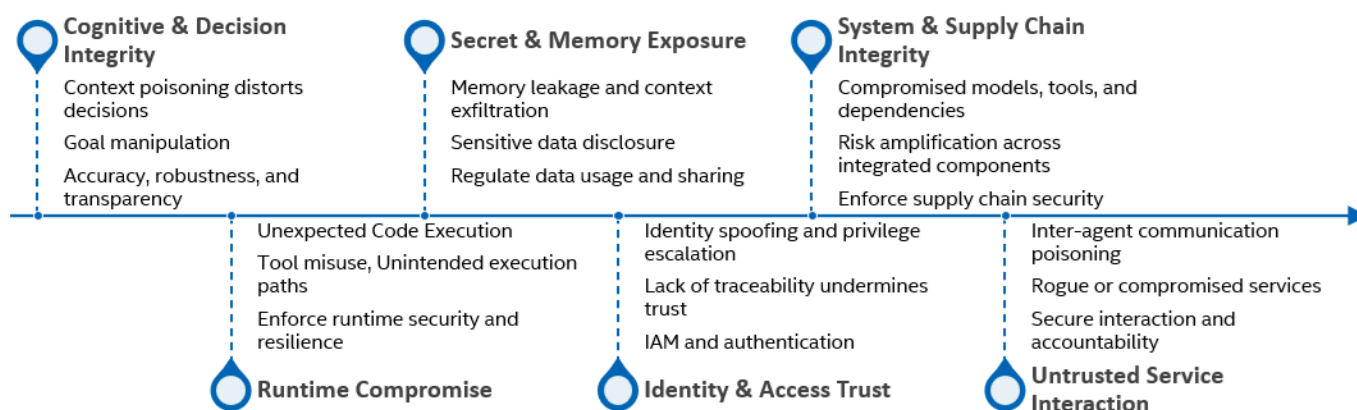
## 1.2 Security Threat Overview

OWASP (Open Worldwide Application Security Project) recently published two key whitepapers for agent security: [Agentic AI – Threats and Mitigations](#) and [OWASP Top 10 for Agentic Applications 2026](#). These two references describe AI and agent security exposure through 17 major threat classes and 10 highest-impact risks. In parallel, per [AI AGENTS UNDER EU LAW A COMPLIANCE ARCHITECTURE FOR AI PROVIDERS](#), EU laws introduce explicit compliance expectations for AI and agent deployment, including accountability, traceability, risk management, transparency, access control, and secure operation.

From a technical perspective, these attack risks, threat patterns, and regulatory requirements can be mapped into six operational categories:

1. Cognitive & Decision Integrity: preserve decision correctness and robustness against context poisoning, goal manipulation, and opaque reasoning behavior.
2. Runtime Compromise: prevent unexpected code execution, tool misuse, and unintended execution paths through hardened runtime controls.
3. Secret & Memory Exposure: reduce memory leakage and context exfiltration, and enforce strict policy over sensitive data usage and sharing.
4. Identity & Access Trust: prevent identity spoofing and privilege escalation with strong authentication, IAM, and auditable identity binding.
5. System & Supply Chain Integrity: detect and contain compromise in models, tools, dependencies, and build-to-runtime software pathways.
6. Untrusted Service Interaction: secure agent-to-agent and agent-to-service communication against poisoned channels, rogue services, and weak accountability boundaries.

Among these six categories, the first is primarily an AI-native security problem rooted in models, prompts, reasoning, and decision behavior. By contrast, Categories 2 to 6 are more directly tied to deployment architecture and runtime trust, across data security, code and runtime integrity, identity, software supply chain, as well as external service interaction. This distinction matters for the discussion that follows: from a deployment perspective, confidential computing can provide concrete security capabilities to strengthen containment, confidentiality, attestation, integrity, and trusted interaction across those latter deployment-facing categories. In the following sections, our discussion focuses mainly on Categories 2 to 6.



**Figure 2: Agent Threats Categories**

## 2. Deployment Models

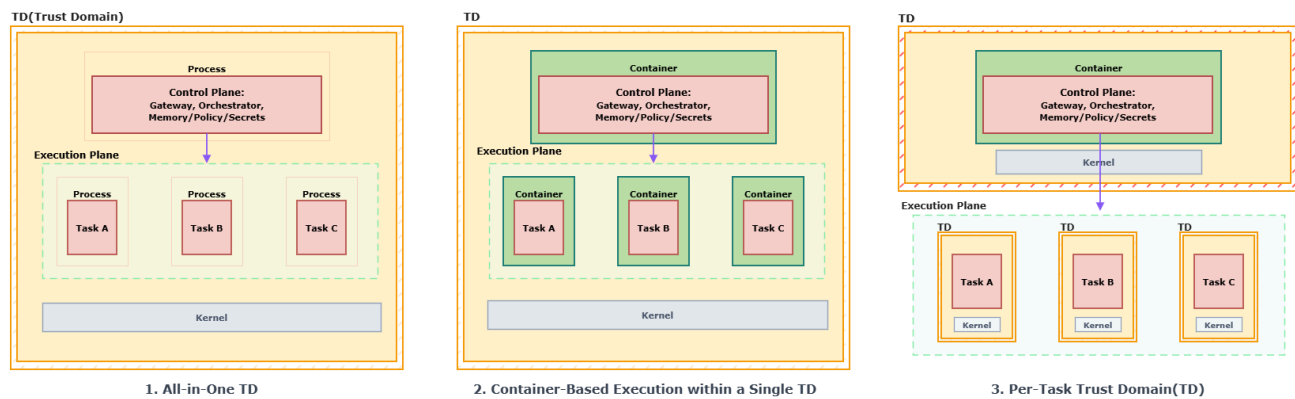
The same agent architecture can be deployed in different ways depending on the required trust boundary, isolation strength, attestation granularity, and operational model. In TDX-based agent deployment, this is best evaluated by how the Control Plane and Execution Plane are split across trust domains and how finely tasks are isolated, which can be compared along four dimensions:

- Isolation strength: how strongly the deployment separates the Control Plane, task workloads, and their surrounding runtime environments, for example through process isolation, container isolation, VM isolation, or TEE-backed isolation.
- Attestation granularity: how precisely the system can produce verifiable claims about what has been launched and what is actually running.

- Operational complexity: how much runtime, orchestration, and image-management overhead is introduced by the deployment model.
- Lifecycle independence: how independently the Control Plane, task workloads, and their images can be deployed, upgraded, and managed.

This document dicusses three representative deployment patterns:

1. All-in-One Trust Domain(TD)
2. Container-Based Execution within a Single Trust Domain(TD)
3. Per-Task Trust Domain(TD)



**Figure 3: Agent Deployment Patterns**

## 2.1 Deployment Patterns

### Pattern 1: All-in-One TD

For this deployment model, the entire agent stack runs as native processes within a single Trusted Domain (TDVM). The TDVM boundary is the system's only strong isolation and the main trust boundary for agent exeuction.

Within the TDVM, both Control Plane components (such as Gateway and Orchestrator) and Execution Plane task workloads run in the same Trust Domain, sharing the same kernel, process namespace, and runtime environment.

This architecture offers the lowest operational complexity and minimal system overhead. It also provides a straightforward attestation model, as the entire agent stack can be treated as a single measured and verifiable unit.

However, the absence of internal isolation means that orchestration logic and task execution share a unified trust domain, without additional containment between components. As a result, any compromise in either component set can impact the entire agent stack. This tightly coupled runtime model also makes it difficult to dynamically upgrade or update dependencies in the execution environment without coordinated changes across the whole stack.

This model is best suited for single-tenant or single-user agent systems, tightly controlled execution environments, or scenarios where the entire TDVM could be treated as a single trust and failure domain.

### Pattern 2: Container-Based Execution within a Single TD

In this model, both Control Plane components and Execution Plane task workloads are packaged as container images and are typically deployed as containers inside a single Trusted Domain.

Container-level isolation is enforced using Linux namespaces and cgroups, providing process-level separation, resource governance, and lifecycle independence between components. In practice, this also provides a lightweight sandboxing effect for execution workloads at the container boundary. However, all containers continue to share the same guest kernel, which serves as the fundamental trust and security boundary of the system.

Compared to the all-in-one monolithic deployment, this pattern introduces a clearer operational and lifecycle separation between control-plane services and task execution workloads. It enables standardized OCI-based packaging, independent image management, and improved deployment scalability across components.

This model is well suited for enterprise-grade internal platforms, standardized cloud-native workflows, and teams that prioritize operational maturity (OCI tooling, CI/CD, image lifecycle management) while not requiring per-task hardware-enforced confidentiality or strong VM-level sandbox isolation.

### **Pattern 3: Per-Task Trust Domain (TD)**

Compared with Pattern 2, the key difference in Pattern 3 is that Execution Plane task workloads run in independent confidential computing sandboxes, rather than sharing a common execution boundary with other components.

In this strongest isolation model, each task executes inside its own confidential computing sandbox (CC sandbox). CC sandbox is an isolated and attestable execution unit for task workloads, with hardware-backed protection for runtime state and memory.

This model is designed to provide:

- High isolation density (many independently isolated task sandboxes per host)
- Fast startup for short-lived or bursty task execution
- Lower runtime overhead than full VM-per-component deployment while preserving hardware-backed isolation

Control Plane deployment can follow two practical layouts:

- TDVM-hosted Control Plane: Control Plane components run in one standard TDVM, and OCI containers are used for packaging, deployment, and verification; task workloads are offloaded to dedicated CC sandboxes.
- CC-sandboxed Control Plane: Control Plane components also run in CC sandboxes, just like task workloads, so each component is independently isolated and attested.

Example implementations include Kata Confidential Containers and other confidential computing-compliant sandbox runtimes. Depending on runtime design, a CC sandbox can be realized through different confidential sandbox mechanisms. Across these deployment layouts and runtime implementations:

- Each task runs in an independently isolated sandbox boundary
- Isolation is enforced at the hardware boundary e.g. Trust Domain
- The sandbox is cryptographically measurable and attestable

This model provides strong containment for untrusted code, third-party tools, and multi-tenant execution, while enabling fine-grained attestation per task. It is best suited for zero-trust execution, sensitive agent workloads, high-density and large-scale deployment environments, and scenarios requiring strict isolation with verifiable execution.

## 2.2 Deployment Pattern Comparison

The three deployment patterns can be compared along two groups of dimensions. The first group focuses on isolation structure: where components run, what boundary separates them, and what runtime model they depend on. The second group focuses on operational and trust-management properties: how images are packaged and updated, how precisely the runtime can be remotely attested, and how much operational flexibility and complexity each pattern introduces.

### 2.2.1 Isolation Strength and Runtime Structure

The table below compares the three deployment patterns from the perspective of component placement, isolation boundary, and runtime structure.

| Feature                    | All-in-One TD                                      | Container-Based Execution within a Single TD     | Per-Task Trust Domain (TD)                                                            |
|----------------------------|----------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------|
| Control Plane placement    | Control Plane components as native processes       | Control Plane components as OCI container(s)     | Control Plane components in main TDVM as OCI container(s), or in their own CC sandbox |
| Execution Plane placement  | Execution Plane task workloads as native processes | Execution Plane task workloads as OCI containers | Execution Plane task workloads in separate CC sandboxes                               |
| Primary isolation boundary | TDVM boundary only                                 | Container boundary inside one TDVM               | Per-task CC sandbox boundary                                                          |
| Shared kernel model        | Shared guest OS and kernel                         | Shared guest kernel across all containers        | Separate guest kernel per sandboxed task                                              |
| Runtime engine             | Native process / systemd                           | Docker / Containerd                              | Kata Confidential Containers or other CC sandbox runtimes                             |
| Task scaling granularity   | Process level within single VM                     | Container level within single VM                 | Per-task CC sandbox                                                                   |

### 2.2.2 Attestation and Operational Properties

Isolation is always a focus, but it is not enough: by itself, it cannot prove runtime trust, cannot resolve malicious issues originating from the images themselves, and cannot prevent compromise risk during operational upgrades. To systematically resolve agent security issues, a production system must also define how images are packaged, how updates are rolled out, at what granularity the runtime can be remotely

attested, and how much operational flexibility the deployment model preserves. The comparison below focuses on those operational and trust-management dimensions.

| Dimension               | All-in-One TD                                                                                    | Container-Based Execution within a Single TD                                                                                                              | Per-Task Trust Domain (TD)                                                                                             |
|-------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Image management        | Single TDVM image; the entire environment is treated as one image and one operational unit       | OCI images for Control Plane components and Execution Plane task workloads; images can be updated independently                                           | Multiple options: Control as OCI container + per-task CC sandbox images, or each component as its own CC sandbox image |
| Attestation granularity | One measurement for the entire TDVM                                                              | TDVM measurement plus image-level identity for containers; can distinguish Control Plane component and Execution Plane task workload images operationally | Per-task CC sandbox measurement; can prove which task image is running in which confidential sandbox                   |
| Flexibility             | Lowest; any significant change typically requires rebuilding or redeploying the whole TDVM image | Moderate; Control Plane component and Execution Plane task workload images can be rolled independently while keeping one TDVM                             | Highest; Control Plane and task sandboxes can evolve more independently, especially in per-task CC sandbox mode        |
| Security                | Single TEE boundary; compromise inside the TDVM affects the whole runtime scope                  | Better packaging discipline, but shared-kernel risk remains                                                                                               | Strongest isolation and strongest separation of trust domains across tasks                                             |

Summary:

- **All-in-One TD** favors simplicity and a single measured trust unit.
- **Container-Based Execution within a Single TD** improves operational flexibility and image lifecycle management, but still depends on a shared guest kernel.
- **Per-Task Trust Domain (TD)** offers the strongest per-task isolation and the finest-grained attestation model, at the cost of higher operational complexity.

The protection model for All-in-One TD has already been analyzed with OpenClaw-CC in [confidential-computing-zoo/openclaw-cc](#), which is the predecessor of Agent-CC. This article mainly focuses on Pattern 2 and Pattern 3, while most of the mechanisms discussed here can also be applied to Pattern 1.

2.4 Security Analysis for Agent Deployment

Across the three deployment patterns, the key issue is how Control Plane and Execution Plane placement affects compromise containment, runtime-state protection, and trust across images and services.

This section focuses on deployment- and operation-related concerns introduced in Section 1.2. The following subsections examine these concerns from two perspectives: first, the protection offered by conventional

isolation mechanisms such as processes, containers, and virtual machines; and second, the additional security properties introduced by TD-based execution.

2.4.1 Conventional Isolation: Capabilities and Limits

Before considering TD-based execution, it is useful to clarify both the value and the limits of conventional isolation. Processes, containers, and virtual machines do provide progressively stronger containment, but the progression is still built around software-enforced boundaries. As a result, three systemic gaps remain: privileged layers can still observe or interfere with in-use state, runtime trust is difficult to prove across trust boundaries, and the resulting boundary is still too weak to serve as a dependable security firewall for AI agent systems.

| Mechanism | What it isolates well                                                                             | Main security limit                                                                                  | Trust implication                                                                                                     |
|-----------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Process   | Limited containment within a single OS process boundary                                           | A privileged host or guest OS can still inspect memory or interfere with execution                   | Very weak runtime proof; trust relies mostly on software identity and access control                                  |
| Container | Better workload separation through namespaces and cgroups, but still bounded by the shared kernel | Containers still share the same kernel, so kernel compromise or container escape can break isolation | Limited; image identity is clearer, but the runtime environment is still difficult to verify strongly                 |
| VM        | Stronger containment through a separate guest kernel and VM boundary                              | The hypervisor or privileged host still remains a major trust dependency in ordinary VMs             | Moderate; measured boot and VM identity are possible, but runtime proof is still relatively coarse and host-dependent |

In short, conventional isolation can narrow blast radius and improve workload separation, but it does not remove reliance on privileged software layers or turn the runtime into a strongly verifiable trust domain. For AI agents, that gap matters because the same system must contain untrusted execution, protect live memory and secrets, and establish trust when interacting with external services. This is the point at which TD-based execution changes the model rather than merely strengthening the same one.

2.4.2 Gains with TD-Based Isolation

TD-based execution should not be treated as just an evolution of conventional isolation; it changes the trust model. Instead of relying mainly on software boundaries, it anchors confidentiality and measurement in hardware-backed execution. For agent systems, this matters because compromise, leakage, false trust claims, supply-chain issues, and unsafe service interaction often reinforce one another. The lack of a verifiable trust foundation is a major contributor to the complexity of today's agent security and gaps across many real-world adoption efforts.

The table below summarizes what TD addresses directly, what still needs complementary controls, and where gaps remain.



| Security concern                | What TD helps do directly                                                                             | What it should be combined with                                                                  | Remaining limitation                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Runtime Compromise              | Reduces host-side exposure and strengthens isolation for control and task execution                   | Task sandboxing, least privilege, policy enforcement, syscall filtering                          | Does not stop malicious logic inside the trusted boundary                                    |
| Secret & Memory Exposure        | Protects in-use memory from host, hypervisor, and operator inspection                                 | Secret scoping, short-lived credentials, output filtering, data minimization                     | Application logic can still leak or misuse secrets                                           |
| Identity & Access Trust         | Provides hardware-backed runtime identity evidence and stronger attestation-based trust establishment | Strong authentication, IAM policy, identity binding, verifier infrastructure, key-release policy | Attestation and identity proof do not guarantee safe behavior or correct authorization logic |
| System & Supply Chain Integrity | Extends trust from signed artifacts to measured boot and launch                                       | Image signing, SBOMs, dependency governance, reproducible builds, CI/CD admission                | Measured software may still contain vulnerabilities                                          |
| Untrusted Service Interaction   | Enables attested channels that verify peer identity and runtime state                                 | TLS, service authorization, trust policy, secure session setup, key management                   | Trusted runtimes can still reach bad peers without policy enforcement                        |

TD is most valuable where both in-use data protection and verifiable runtime trust are required. In this model, TD serves as the trust foundation by providing hardware-rooted memory protection and measurable runtime identity across trust boundaries.

To be clear, TDX-based deployment is not a silver bullet, but it makes agent security challenges easier to address through structured and verifiable controls. It provides a trust foundation, not a complete security architecture for agent systems. It should be combined with sandboxing, policy enforcement, signed images, encryption at rest, and TLS to systematically reduce deployment-facing threats.

### 3. Protecting Agent Data Across Its Lifecycle

AI data security risks are amplified in agent scenarios because processing extends beyond a single request. Agents continuously take prompts, pull external context, generate intermediate state, retain user memory, and may persist results. Sensitive data therefore flows through an end-to-end pipeline during reasoning, execution, and persistence, making data-in-use protection critical. If in-use data is exposed at any stage of this pipeline, the agentic AI system directly faces data security risk. For agentic AI systems, runtime compromise and secret or memory exposure are therefore central risks.

#### 3.1 Why Data Lifecycle Protection Matters

In a typical agent workflow such as OpenClaw, data flows through multiple stages: user prompts and context enter via a gateway, orchestration and planning components pull external knowledge, execution runtimes invoke tools and APIs with credentials, and memory services persist conversation history and retrieved

artifacts. At each point in this flow, sensitive data faces distinct threats, and the highest-impact risks occur during data-in-use and data-at-rest.

The most critical stage is data-in-use. In today's cloud environments, protections for data-at-rest and data-in-transit are relatively mature. By contrast, data-in-use protection is still typically the weakest layer. When runtime context, intermediate reasoning state, permission tokens, and tool outputs coexist in active memory, runtime compromise, high-privilege infrastructure observation, or memory leakage can directly expose the full working context and active credentials at once. This is where adversaries can cause maximum damage. Protecting data-in-use must therefore be the top priority in agent security design.

For agents, conversation history, retrieved artifacts, long-term memory, and audit traces are continuously accumulated and reused, making data-at-rest a persistent and high-value attack surface.

Data ingress and exchange add further exposure: poisoned prompts, weak identity checks, and over-privileged credentials can compromise the pipeline at its edges. Protecting the full lifecycle is therefore a core design requirement, because failure at any stage propagates downstream.

## 3.2 Intel TDX-Based Protection Mechanisms

TD serves as the trust foundation here: it provides hardware-rooted memory protection and measurable runtime identity that the data protection mechanisms in this chapter depend on. Agent processes run inside a hardware-isolated boundary where memory is always encrypted, shielded from the host, hypervisor, and other privileged software layers. Runtime identity and launch measurements are verified through attestation, and secrets are released only to attested environments. Encrypted storage is decrypted only inside TD-verified runtimes, extending that same isolation and trust from data-in-use through to data-at-rest.

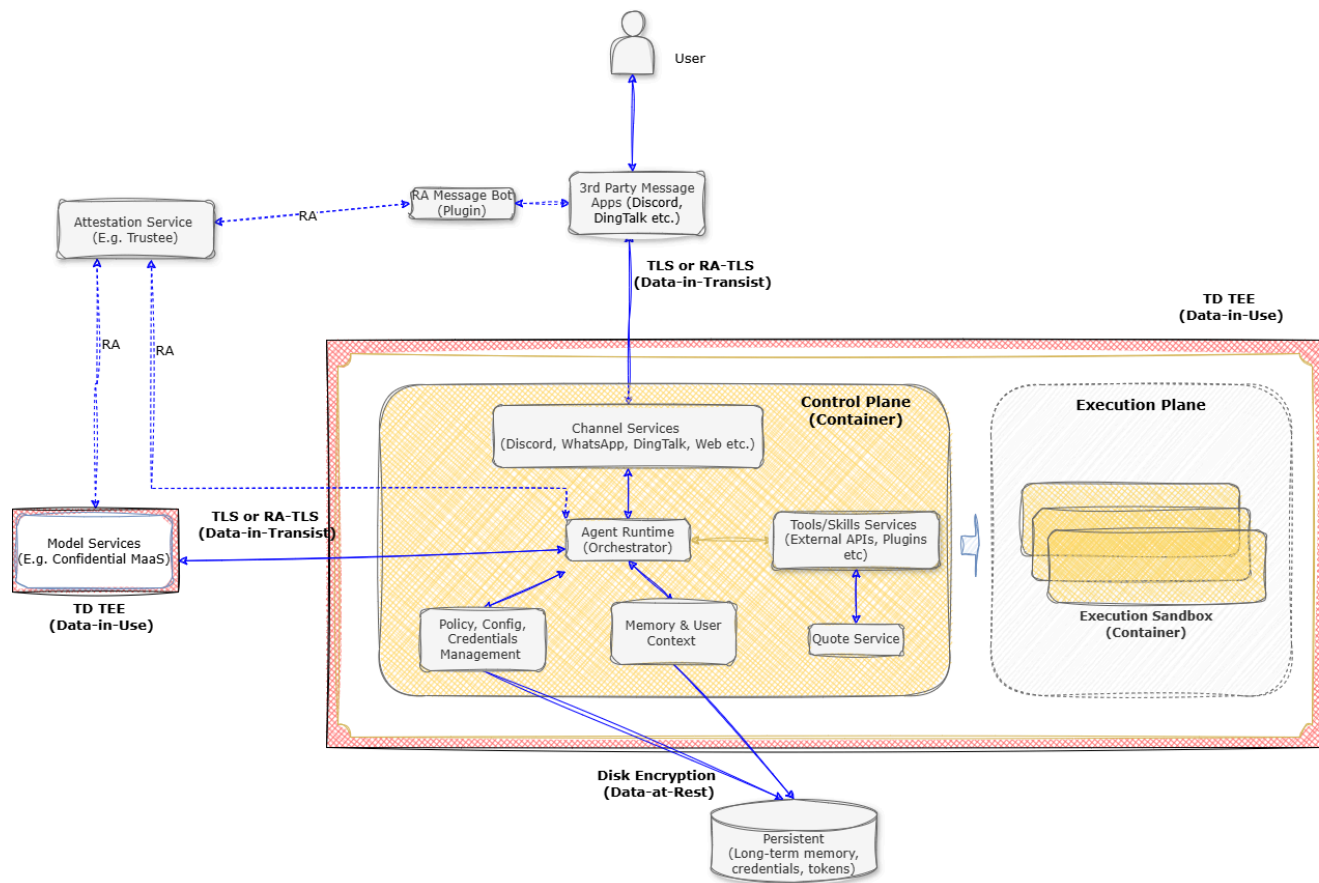
The deployment model addresses three critical data surfaces with distinct TDX-backed protections:

**Runtime Context and Memory (Data in use):** Runtime context, intermediate reasoning state, and tool outputs coexist in active memory during execution. Without protection, this data faces exposure through high-privilege infrastructure observation or memory leakage. TDX mitigates this risk by ensuring core processes and services run inside a TDX-based Trusted Execution Environment where memory is always encrypted. Combined with runtime integrity verification and dynamic attestation, this prevents unauthorized memory inspection and provides continuous assurance that the executing workload remains trusted.

**Secrets and Permissions (Data in use):** API keys, OAuth tokens, and access credentials are high-impact assets that enable privilege abuse and cross-service breaches if compromised. TDX confines credential usage strictly within the TEE boundary. Credentials are never exposed to the host or untrusted layers; instead, remote attestation gates secret release, ensuring that keys are provisioned only to verified, trusted environments. This transforms the threat model from "credential exposed after release" to "credential released only after trust verification."

**Long-Term Memory and Persisted Data (Data at rest):** Conversation history, retrieved artifacts, long-term memory, and audit traces accumulate high-value sensitive data over time. Storage and transmission introduce exposure windows. TDX-backed designs address this by encrypting data at rest and in transit, with decryption occurring only within the TEE. Sensitive data access is further managed through remote attestation: stored data is released only to approved runtimes that have passed attestation checks, creating a unified encryption and access control model across persistence boundaries.

This integrated approach combines in-use protection with at-rest safeguards. TDX protects active memory and gates secret release through attestation, while encryption and attestation-based access control protect persisted state. The result is a coherent protection path where sensitive data is encrypted by default, decrypted only inside verified TEE boundaries, and released to external services only after attestation succeeds.



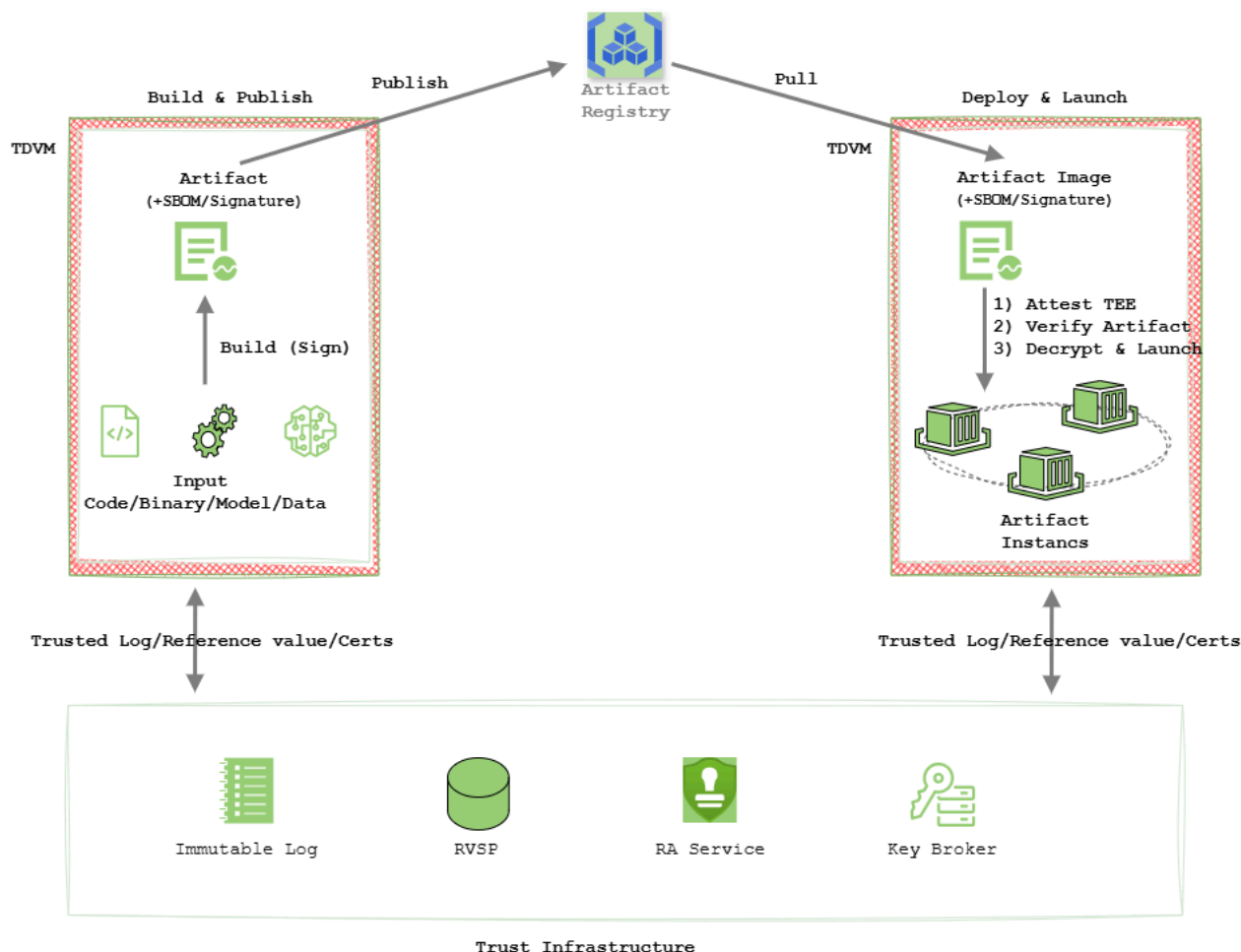
**Figure 4: Full data lifecycle protection - OpenClaw as example**

In agentic AI, memory is an emerging domain and a core trust component for continuity, personalization, and decision quality over time. Today, most of today memory mechanisms are still implemented as agent-local state. The next major innovation, however, is serviceized memory that supports cross-agent and cross-domain sharing and evolution. As memory moves in that direction, the same data protection model discussed in this chapter remains applicable to memory services: confidential execution for in-use context, attestation-based identity and admission, policy-gated secret and key release, and encrypted, policy-controlled persistence and exchange.

## 4. From Build Artifacts to Attested Execution

This chapter responds directly to the security concerns introduced earlier by focusing on the requirements behind them. Those concerns create a practical need to prove what is actually running, detect whether the runtime has deviated from approved software and policy, and provide verifiable execution claims to other system components, verifiers, or peer services. In agent systems, it is not enough to know which software was built or which deployment policy was intended. To enforce those security goals in practice, the system must be able to prove that the running agent is actually executing inside the expected trust boundary, with the expected measurement, software identity, and policy state.

This is why a connection between build artifacts and attested execution is necessary. Without that connection, signed images, approved configurations, and deployment policy remain only build-time intent, while the runtime may still differ from what the platform believes it launched. In the agent deployment scenario, the practical question becomes: how does a deployed agent workload become a trustworthy runtime?



**Figure 5: From Build Artifacts to Attested Runtime**

#### 4.1 Packaging, Measurement, and Launch Boundary

The three deployment patterns differ not only in isolation strength, but also in what is packaged, what is measured at launch, and what a verifier is ultimately asked to trust. At the same time, they share one common structure: in every pattern, there is something the system delivers as a deployable artifact, something the TEE actually measures at launch, and some final runtime unit that the verifier is asked to trust.

| Pattern                                      | Main packaging unit            | Main measured unit                                              | Final trust unit                            | Attestation granularity |
|----------------------------------------------|--------------------------------|-----------------------------------------------------------------|---------------------------------------------|-------------------------|
| All-in-One TD                                | one bootable TDVM image        | kernel, initrd, rootfs, launch configuration                    | one TDVM containing Control Plane and Tasks | whole system            |
| Container-Based Execution within a Single TD | one TDVM image plus OCI images | TDVM boot artifacts, optionally plus runtime-measured OCI state | one measured TDVM                           | guest / TDVM level      |

| Pattern                    | Main packaging unit                                      | Main measured unit                                                   | Final trust unit | Attestation granularity |
|----------------------------|----------------------------------------------------------|----------------------------------------------------------------------|------------------|-------------------------|
| Per-Task Trust Domain (TD) | control image plus one sandbox image or payload per task | control-domain boot artifacts plus each task sandbox launch artifact | per-task sandbox | task / sandbox level    |

This distinction is fundamental across all three patterns. The packaging unit may be a bootable TDVM image, an OCI image, or a sandbox payload, depending on how the workload is delivered. The measured object, however, is the launch path that enters the TEE trust chain: typically kernel, initrd, payload or root filesystem, and launch configuration. In the simplest case, these layers are close to each other: one bootable image is delivered, one guest is launched, and one measured trust unit results. In the more modular cases, they begin to separate: the platform may deliver OCI images or per-task sandbox payloads, while the verifier still needs evidence about the concrete measured runtime that was actually launched. In OCI-based cases, runtime measurement can later extend the chain by recording OCI digests, policy state, or launch decisions into attestation-visible state such as RTMR, but that does not change the basic point: packaging identity and measured identity are related, not identical.

The launch path explains what the attestation claim naturally means in each pattern.

- **All-in-One TD** is the most direct case: one TDVM image is delivered, one guest system is launched, and one measured trust unit is produced. The resulting claim is simple: one confidential VM booted with the expected system image and configuration.
- **Container-Based Execution within a Single TD** keeps the same primary hardware trust anchor, because one TDVM is still launched and measured first. The difference is that the delivered workload now includes OCI images inside that guest. This adds an extra layer between packaging identity and measured launch identity: the verifier trusts the measured TDVM directly, while OCI identity must be enforced or recorded inside that trusted guest.
- **Per-Task Trust Domain (TD)** moves the trust boundary closer to the workload, but it still does not measure only the workload in isolation. The Control Plane may still run in a TDVM or containers, while each task launches into its own confidential sandbox with its own boot artifacts, such as guest kernel, initrd, root filesystem or base payload, and then the task-specific workload is bound into that sandbox, often through an OCI-based path. The important difference from the earlier patterns is not the absence of a guest-style launch base, but that the final trust unit is narrowed to an individual task sandbox rather than a broader shared guest environment.

The main architectural consequence is straightforward: VM-style launch tends to produce broader, guest-centric trust claims, while MicroVM- or sandbox-style launch makes it easier to produce workload-specific trust claims. Put differently, the closer the measured launch boundary is to the workload itself, the more precise the resulting trust claim can be.

## 4.2 Remote Attestation and Trust Claims

Remote attestation is the step that converts measured launch evidence into a trust decision. In all three patterns, the verifier compares runtime evidence against expected build identity and launch policy. What changes is the granularity of the resulting claim.

- In **Pattern 1**, the verifier checks one expected TDVM measurement against one attestation report.

- In **Pattern 2**, the verifier first establishes trust in the measured TDVM, then evaluates whether OCI image identity was correctly enforced or recorded inside that trusted guest.
- In **Pattern 3**, the verifier can evaluate the Control Plane and each task sandbox separately. The task-side claim is based on the measured sandbox plus the binding evidence that links the intended workload identity to that sandbox.

This also determines when secrets should be released. Secret release should happen only after three things align: the measured launch matches policy, the intended workload identity is known, and the system can prove that the workload identity is correctly bound to that measured launch.

The core point is that agent deployment with confidential computing is not only about launching a TEE, but about turning runtime evidence into a trustworthy claim. What changes across the three patterns is the granularity of that claim, from whole-system trust, to guest-level trust with in-guest workload identity, to per-sandbox trust with tighter workload binding.

### 4.3 Binding Agent Workload to Measured Runtime

Pattern 1 treats the whole guest as one trust unit: the system image is built, measured, and attested as a single object. Pattern 2 and Pattern 3 change that model. In both cases, the measured TEE launch base and the delivered workload are no longer the same artifact. The runtime is built from at least two layers: a measured guest or sandbox base, and a separately delivered agent workload, often packaged as an OCI image.

That separation creates the core problem of this chapter. It is no longer enough to prove that a confidential guest or sandbox base booted correctly. The system must also prove that the expected agent workload is the one actually bound to that measured runtime. This document refers to that proof path as the trusted binding layer. In this sense, Pattern 2 and Pattern 3 face the same basic problem: both must connect workload identity to a measured TEE runtime. The main difference is where that binding layer sits and how it is implemented.

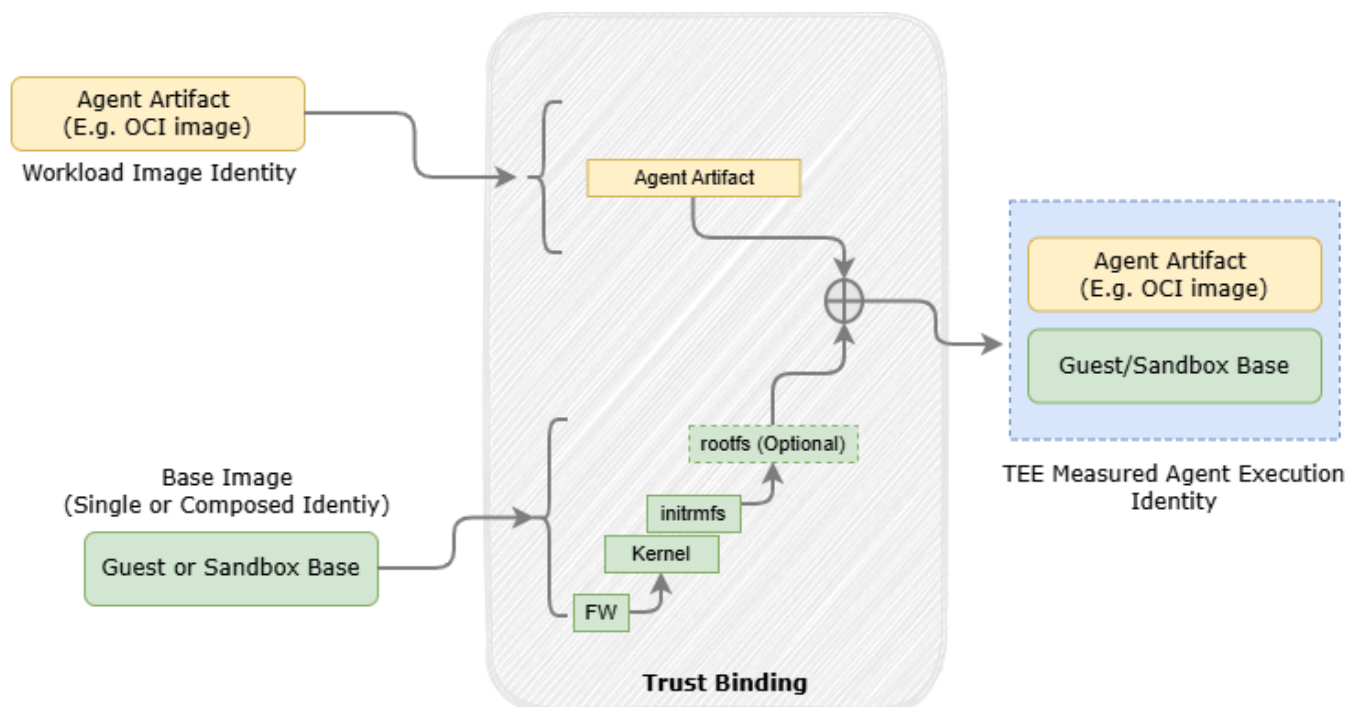
#### The Trusted Binding Layer

The trusted binding layer is the minimum trusted software path that connects workload identity to TEE-measured execution identity. In many practical systems that workload identity is carried by an OCI image, but the key issue is broader than OCI alone. Its purpose is not merely to launch a workload, but to preserve a verifier-usable relationship between what was requested, what was measured, and what is allowed to receive secrets.

At a minimum, this layer should do five things:

1. **Check the image:** verify that the expected OCI image is the one being used.
2. **Bind image to runtime:** make sure that image is the one actually loaded into the confidential environment.
3. **Expose it to attestation:** ensure the workload identity can be seen in attestation evidence.
4. **Apply policy:** enforce the required image, configuration, and launch rules.
5. **Gate secrets:** release secrets only after both the runtime and workload identity are verified.

In Pattern 2, this trusted binding layer can often be realized by the guest-side workload runtime path inside the measured TDVM. In Pattern 3, it is often realized by a TEE-facing trusted shim or trusted agent that performs the same binding function for a separately measured task sandbox.



**Figure 6: Trust Binding Layer**

## Binding Styles and Verification

There are two main ways to bind OCI identity to confidential execution. In both cases, the verifier needs evidence that the requested workload identity was correctly associated with a measured confidential runtime. In Pattern 2, that proof is usually produced inside the measured TDVM. In Pattern 3, it is usually produced at the boundary of a separately measured task sandbox, even if the orchestration path begins outside the sandbox TCB.

### Pre-boot binding

In the first style, the OCI image is translated into the sandbox root filesystem or payload before launch. The workload identity is then absorbed into the boot-time measured artifact.

This gives the strongest direct linkage between application identity and boot measurement, because the verifier can reason about one integrated measured payload. The trade-off is operational flexibility: updating the workload may require rebuilding or regenerating the sandbox payload.

### Post-boot runtime binding

In the second style, the sandbox base is launched first, and OCI-related identity is recorded after boot through runtime measurement, for example by extending image digests, configuration, or policy state into RTMR.

This is where TDX dynamic measurement is especially useful for post-boot runtime binding. It allows runtime evidence to be appended in a hardware-rooted way after launch, so the verifier can evaluate both the measured launch base and the later binding evidence in one attestation flow.

This keeps OCI-style workflows more flexible, but it also makes verifier logic more demanding. The verifier must now reason over both the initial sandbox measurement and the later runtime measurements that bind workload identity to that sandbox.

In practice, many confidential-container systems use a hybrid form of this second model: `containerd` continues to manage OCI lifecycle, while a trusted shim on the TCB-facing side translates OCI intent into a measured sandbox launch and records the binding evidence needed for attestation.

Regardless of implementation style, a verifier should not accept a claim based only on guest or sandbox boot integrity. For OCI-delivered confidential workloads, three checks are required:

1. **Measured base integrity:** the TDVM or sandbox boot artifacts match expected policy.
2. **Expected OCI identity:** the requested image digest and relevant configuration match deployment intent.
3. **Trusted binding evidence:** the system proves that this OCI identity is the one actually associated with the measured confidential runtime.

Only after these three checks succeed should the relying party treat the workload as eligible for trusted execution, service admission, or secret release, because a credible workload-specific trust claim depends on explicitly binding the agent workload identity to the measured TEE runtime.

## 5. Trusted Service Composition

This chapter focuses on trust continuity across service boundaries. In agent systems, security is not decided only by how the local runtime is protected, but by how the platform decides which external services are allowed to receive sensitive requests. An agent may run in a trusted runtime yet still leak risk through downstream calls to model services, retrieval systems, memory backends etc.. The core problem here is therefore not only channel security, but service admission under runtime trust evidence: the platform must decide when a remote service is trustworthy enough to participate in the same sensitive processing path.

### 5.1 External Service Interaction

In practice, agent security must extend from trusted local execution to policy-governed trusted service access. If a retrieval service, or memory backend runs in an untrusted environment, then data may still be exposed, modified, or mishandled even though the originating agent runs in a TEE. The practical requirement is therefore stronger than transport encryption alone. It is not enough to know which service identity the agent is connecting to; the system should also know whether that service is running in an expected and verifiable execution environment.

TEE-based deployment helps address this problem by extending confidential execution and remote attestation across service boundaries. External services that handle sensitive agent data should themselves run inside TEE-isolated environments, so that data remains protected not only inside the agent runtime but also during downstream processing. Combined with attested channels, this allows the agent side or relying platform to verify both service identity and service runtime state before sending sensitive requests. In this model, the security claim is not merely that the agent is trusted, but that the full processing path from agent to external service remains within measured, policy-approved execution boundaries.

### 5.2 Trust-Governed Service Access

The architectural implication is that service admission should be conditioned on trusted proof, much like secret release is conditioned on attestation. In this document, that verifier-mediated admission function is referred to as a Trust Guard: a control-plane enforcement layer that validates attestation evidence, binds it to service identity and policy, and permits access only when those checks succeed. Together, these checks build



an end-to-end trust chain from the agent runtime, through the communication channel, to the external service runtime. Before an agent sends sensitive prompts, memory context, or credentials to an external service, the platform should verify at least three things: the service identity is the intended one, the service runtime is executing inside an expected TEE environment, and the attested software and configuration state satisfy policy. This allows the platform to treat service access as a verifier-controlled trust decision rather than a simple network connection.

When this is combined with encrypted transport, the result is a more complete protection path for agent data. TEE isolation protects data while it is processed inside the agent and inside the external service. TLS or a similar secure channel protects data in transit between them. Remote attestation then ties both endpoints to verifiable runtime state, so that confidentiality and integrity do not depend only on network identity. This is the practical meaning of end-to-end protection in the agent setting: sensitive data remains protected from the moment it leaves the agent runtime until it is processed inside an attested external service runtime.

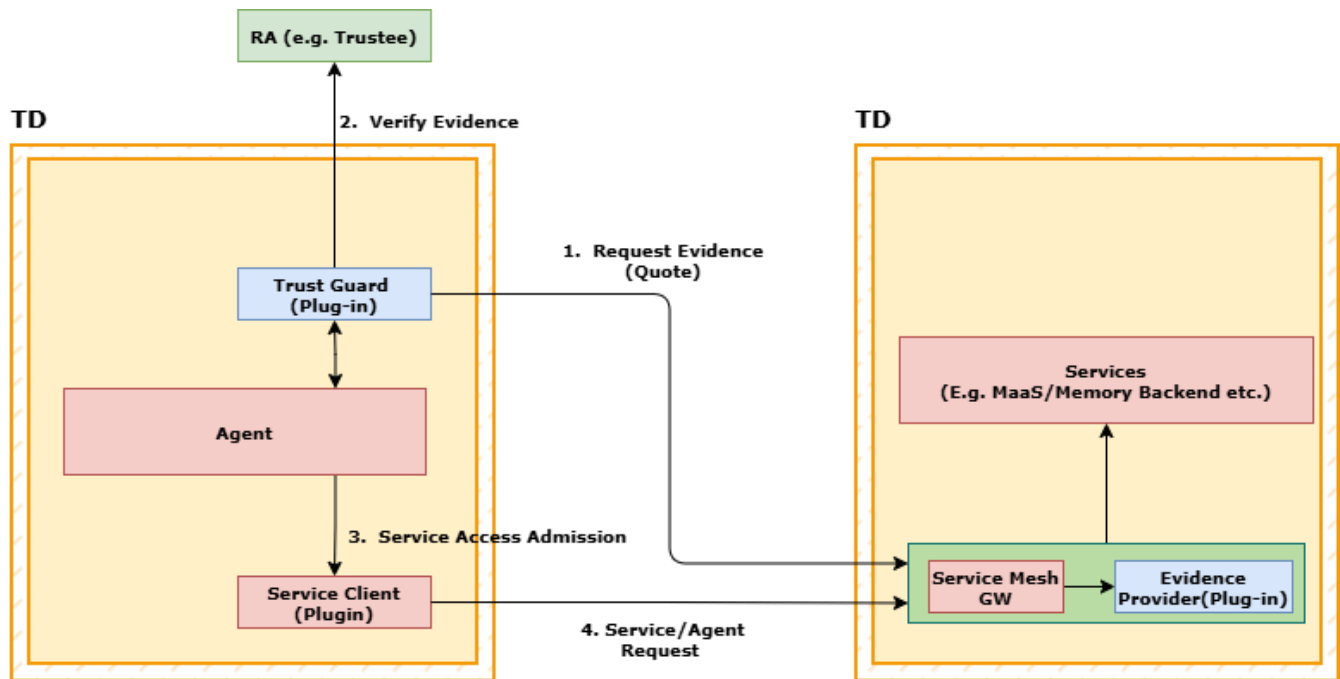
### 5.3 Non(or Low)-Intrusion Integration

In conventional service deployments, adding remote attestation directly into each agent and service is usually high-friction. Teams must introduce quote generation and verification logic, nonce/challenge handling, policy synchronization, identity binding, and failure-path handling across many components. This increases implementation cost, slows release cycles, and makes multi-service operations harder to manage.

To preserve a lift-and-shift operating model, Agent-CC adopts a low-intrusion integration principle: do not require structural changes to agent or service business logic, and do not force API redesign for existing services. Instead, remote attestation and trust decisions are moved to the deployment control plane, where connectivity and admission are already governed.

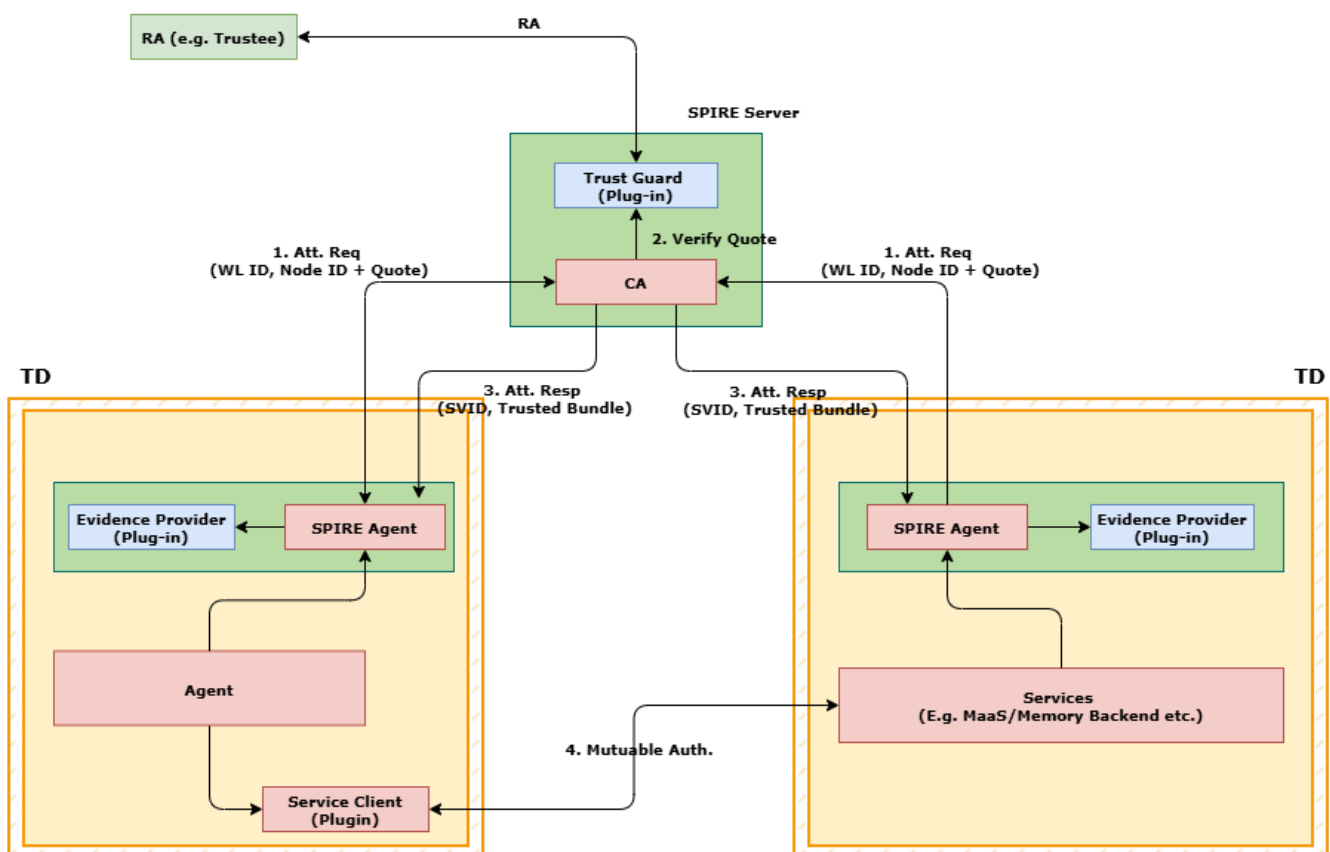
In practice, this control-plane integration can be realized through service mesh and gateway layers (for example, Envoy- or Nginx-based sidecar/gateway paths) and through identity systems (for example, SPIRE/SPIFFE-based workload identity paths). These layers verify TDX evidence, map measurements to workload identity and policy, and enforce allow/deny decisions before service-to-service communication or credential issuance.

Envoy, Nginx, and SPIRE/SPIFFE are representative examples in this document, not mandatory implementation choices. Any equivalent gateway, sidecar, or workload-identity framework that can enforce attestation-aware policy decisions and bind identity to verified runtime evidence can be used.



**Figure 7: Trust-Guard Architecture in a Service Mesh.**

This approach keeps deployment complexity manageable while extending trust across the agent-service interaction path. More broadly, TDX and confidential computing should be treated as an enabling layer: they do not replace model-level, policy-level, or application-level defenses, but they provide a verifiable trust foundation that can be extended to address more general agentic security threats across runtime compromise, data exposure, identity abuse, supply-chain drift, and untrusted service interaction. In this way, the proposed Agent-CC architecture uses TDX not only to protect a single agent runtime, but to establish a practical confidential-service fabric in which both the agent and the external services it depends on can be verified as trustworthy participants.



**Figure 8: Trust-Guard architecture based on SPIRE model.**

6. Conclusion

Taken together, this document presents a practical TDX-based Agent Confidential Computing reference architecture that addresses the major deployment-relevant threat categories through a unified trust model. Chapters 3 to 5 describe this model in three layers: lifecycle data protection, build-to-runtime integrity, and trusted service interaction.

The resulting architecture provides three system-level capabilities:

- 1. **Lifecycle data protection:** protect in-use data inside confidential runtimes, secure persisted state with encryption at rest, and gate access through attestation and policy.
- 2. **Build-to-runtime integrity:** connect build intent, launch measurements, workload-binding evidence, and verifier decisions into an enforceable trust chain.
- 3. **Trusted service composition:** enforce identity- and evidence-based admission so data flows only to approved services.

These five threat categories are not eliminated in an absolute sense, but they can be materially reduced or better contained by an Agent-CC design built on TDX.

| Threat category                    | How Agent-CC on TDX helps reduce it                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Runtime<br>Compromise              | Moves the agent runtime into a measured and isolated execution boundary, reducing host-side attack exposure and strengthening containment.                                                                           |
| Secret & Memory<br>Exposure        | Protects live memory with hardware-backed isolation, gates secret release on attestation, and can be combined with encrypted persistent storage for durable state.                                                   |
| Identity & Access<br>Trust         | Strengthens attestation-backed identity establishment and trust decisions so access control can rely on verifiable runtime identity rather than on software identity alone.                                          |
| System & Supply<br>Chain Integrity | Connects approved build artifacts to measured launch state and workload-binding policy, making unauthorized runtime deviation easier to detect.                                                                      |
| Untrusted Service<br>Interaction   | Extends trust from the agent runtime to external services through TEE-based service deployment, attested channels, and mesh or identity layers that verify service runtime state before sensitive data is exchanged. |

The key conclusion is that TDX enables an Agent-CC baseline in which confidentiality, integrity, and trust can be evaluated from runtime evidence instead of assumed from infrastructure position. This foundation does not make agent logic automatically safe, nor does it replace existing security strategies and controls. Instead, it shifts the security baseline: TDX and confidential computing provide the enabling layer and trust foundation for agentic AI deployment, and only with this foundation in place can complementary technologies be combined to build agentic AI systems that are truly trustworthy, secure, scalable, and extensible.

7. Reference Repository

As a companion to this document, the Agent-CC GitHub repository, available at [confidential-computing-zoo/agent-cc](#), serves as the practical workspace for the deployment architecture. It brings together design

materials, reference deployments, and prototype implementation components that help translate these concepts into deployable confidential-computing patterns on Intel TDX. Some of the definitions and implementation elements described in this document are planned for release in later versions of the repository.